

Assignment_1

September 14, 2026

1 EN3160 Assignment 1 on Intensity Transformations and Neighborhood Filtering

September 2, 2026

Name: Induwara M.L.A.S. · Index No.: 230261F · GitHub: github.com/SasithIM/EN3160_IP_MV_Files

```
[151]: import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt

%matplotlib inline
plt.rcParams["figure.figsize"] = (10, 4)
plt.rcParams["image.cmap"] = "gray"
plt.rcParams["savefig.bbox"] = "tight"
plt.rcParams["savefig.pad_inches"] = 0.05
```

1.1 Question 1

Implement the intensity transformation depicted in Fig. 1a on the image shown in Fig. 1b by writing a function `intensity_transform(im, breakpoints)` where the break points are, e.g.,

$$\begin{bmatrix} 0 & 0 \\ 50 & 50 \\ 100 & 150 \\ 150 & 255 \\ 255 & 255 \end{bmatrix}.$$

Experiment with different break points to get a visually pleasing output. Show the intensity transformation as a plot and the original and transformed images side-by-side.

(a) Intensity transformation.

(b) Image for intensity transformation.

```
[152]: bp = np.array([[0,0], [50,50], [50,100], [150, 255], [150, 150], [255,255]])

def create_lut(bp):
    """ Create a lookup table for a piecewise linear transformation defined by breakpoints bp. """
    c = np.asarray(bp, dtype=float)
    assert c[0,0]==0 and c[-1,0]==255, "Breakpoints must start at 0 and end at 255"
    t = np.zeros(256, dtype=np.uint8)
    for (x0, y0), (x1, y1) in zip(c[:-1], c[1:]):
        x0, y0, x1, y1 = map(int, (x0, y0, x1, y1))
        if x1 == x0:
            t[x0] = y1
            continue
        t[x0:x1+1] = np.linspace(y0, y1, x1-x0+1)
    return np.round(t).clip(0,255).astype(np.uint8)

def intensity_transform(img, t):
    """ Apply a piecewise linear intensity transformation to an image using breakpoints bp. """
    lut = create_lut(bp)
```

```

return cv.LUT(img, lut)

img = cv.imread("figures/fig1b.png", cv.IMREAD_GRAYSCALE)
t = create_lut(bp)
out = intensity_transform(img, t)

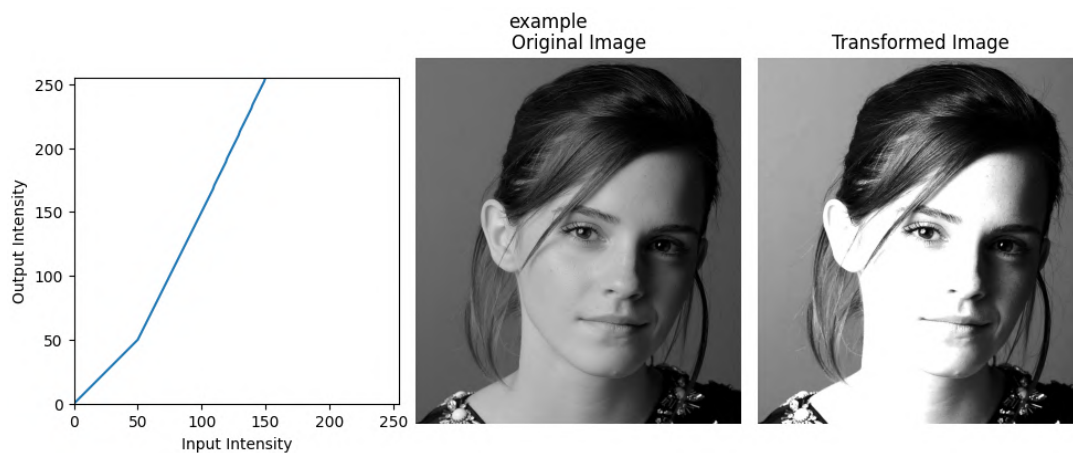
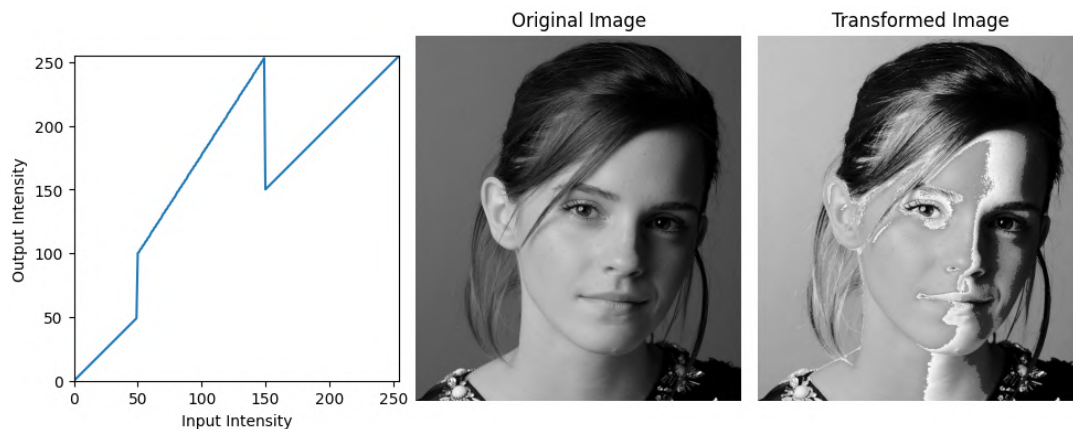
def img_draw(img, out, t):
    fig, axes = plt.subplots(1, 3)
    axes[0].plot(t); axes[0].set_xlabel("Input Intensity"); axes[0].set_ylabel("Output Intensity"); axes[0].
    set_xlim(0, 255); axes[0].set_ylim(0, 255); axes[0].set_aspect("equal")
    axes[1].imshow(img, cmap="gray", vmin=0, vmax=255); axes[1].set_title("Original Image"); axes[1].axis("off");
    axes[2].imshow(out, cmap="gray", vmin=0, vmax=255); axes[2].set_title("Transformed Image"); axes[2].
    axis("off")

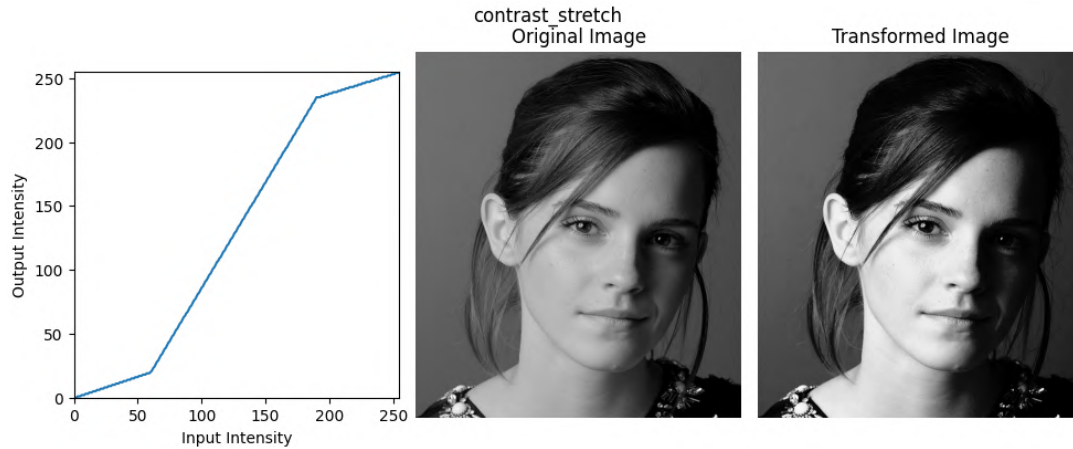
img_draw(img, out, t)
plt.tight_layout(); plt.show()

exp = {
    "example": [[0,0], [50,50], [100,150], [150,255], [255,255]],
    "contrast_stretch": [[0,0], [60,20], [190,235], [255,255]],
}

for j, (name, bp) in enumerate(exp.items()):
    t = create_lut(bp)
    out = intensity_transform(img, t)
    img_draw(img, out, t)
    plt.suptitle(name)
    plt.tight_layout(); plt.show()

```





1.2 Question 2

Apply a similar operation as above (question 1) to accentuate

- (a) white matter
- (b) gray matter

in the brain proton density image shown in Fig. 2. Show the intensity transformations as a plots.

Figure 2: A brain proton density slice.

```
[153]: img = cv.imread("figures/fig2.png", cv.IMREAD_GRAYSCALE)

h = np.bincount(img.ravel(), minlength=256).astype(float)
hs = np.convolve(h, np.ones(9)/9, mode='same')
# modes
peaks = [i for i in range(6, 250)
         if hs[i] == hs[i-6:i+7].max() and hs[i] > 0.002*img.size]
print('peaks:', peaks)                                # 7, 168, 195, 228

# valleys between consecutive modes -> class boundaries
for a, b in zip(peaks[:-1], peaks[1:]):
    print(f'valley {a}-{b}: {a + int(np.argmin(hs[a:b+1]))}') # 184, 221

# low edge of the darker mode: 20% of peak height
p = 168
lo = p
while lo > 20 and hs[lo] > 0.20*hs[p]:
    lo -= 1
print('low edge:', lo)

def band_breakpoints(lo, peak, hi, floor=0, ceil=255):
    return [[0, floor], [lo, floor], [peak, ceil], [hi, floor], [255, floor]]

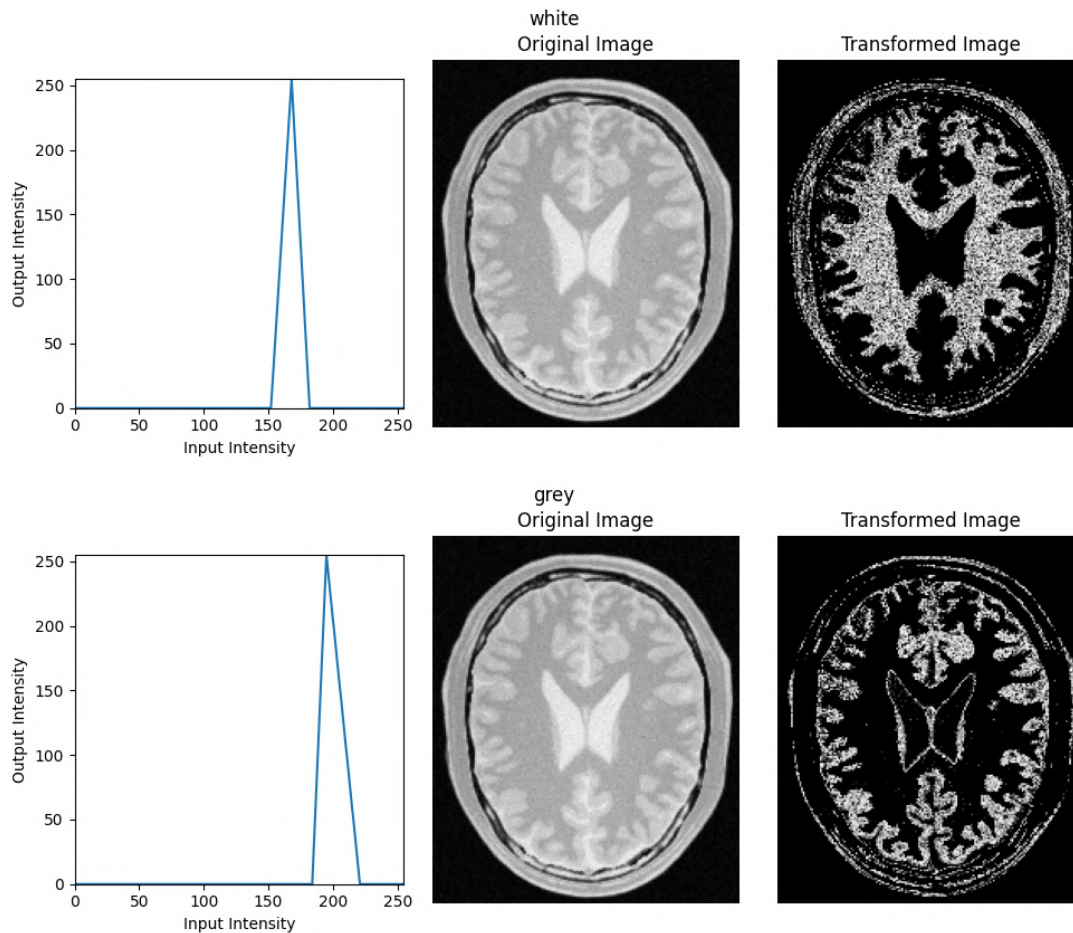
bp_a = band_breakpoints(152, 168, 182) # mode at 168
bp_b = band_breakpoints(182, 196, 213) # mode at 195

transforms = {
    "white": band_breakpoints(152, 168, 182),
    "grey": band_breakpoints(184, 195, 221),
}

for j, (name, bp) in enumerate(transforms.items()):
    t = create_lut(bp)
    out = intensity_transform(img, t)
    img_draw(img, out, t)
    plt.suptitle(name)
```

```
img_show()
```

```
peaks: [7, 168, 195, 228]
valley 7-168: 134
valley 168-195: 184
valley 195-228: 221
low edge: 150
```



1.3 Question 3

Consider the image shown in Fig. 3.1

- (a) Apply gamma correction to the L plane in the $L^*a^*b^*$ color space and state the γ value.
- (b) Show the histograms of the original and corrected images.

1 <https://www.adobe.com/creativecloud/photography/discover/highlights-and-shadows.html>

Figure 3: Image for gamma correction.

```
[154]: img = cv.imread("figures/fig3.png", cv.IMREAD_COLOR)

def gamma_lut(gamma):
    """ create a lookup table for gamma correction """
    return np.array([(i/255)**gamma * 255 for i in range(256)], dtype=np.uint8)

def gamma_correction(img, gamma_lut):
    """ Apply gamma correction to an image using a lookup table. """
    lab = cv.cvtColor(img, cv.COLOR_BGR2LAB)
    l, a, b = cv.split(lab)
    l = cv.LUT(l, gamma_lut)
    lab = cv.merge([l, a, b])
    return cv.cvtColor(lab, cv.COLOR_LAB2BGR)
```

```

gammas = [ 0.5, 0.6, 0.7, 1.0, 1.5]
fig, axes = plt.subplots(2, len(gammas), figsize=(16, 6))

for j, gamma in enumerate(gammas):
    lut = gamma_lut(gamma)
    out = gamma_correction(img, lut)
    axes[0,j].imshow(cv.cvtColor(out, cv.COLOR_BGR2RGB)); axes[0,j].set_title(f'$\\gamma$ = {gamma}'); axes[0,j].
    axis('off')
    axes[1,j].plot(gamma_lut(gamma)); axes[1,j].set_xlim(0,255); axes[1,j].set_ylim(0,255); axes[1,j].
    set_aspect('equal')

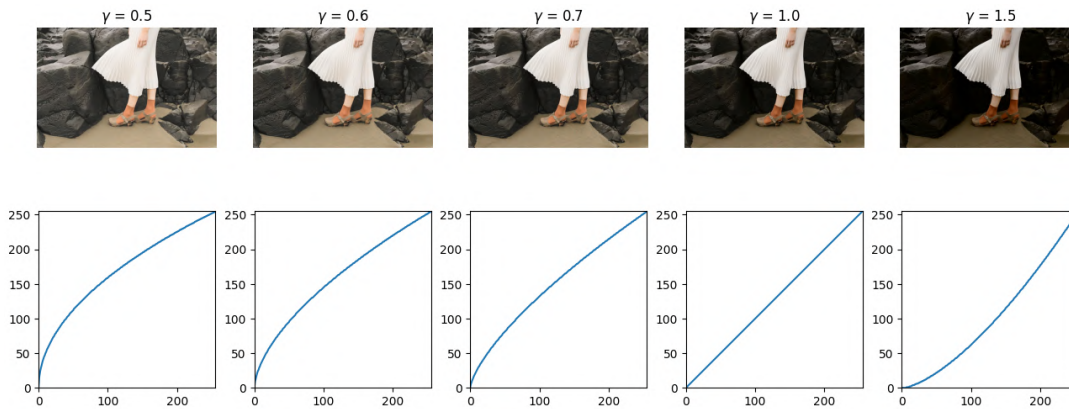
#choosing gamma=0.7 for the final image

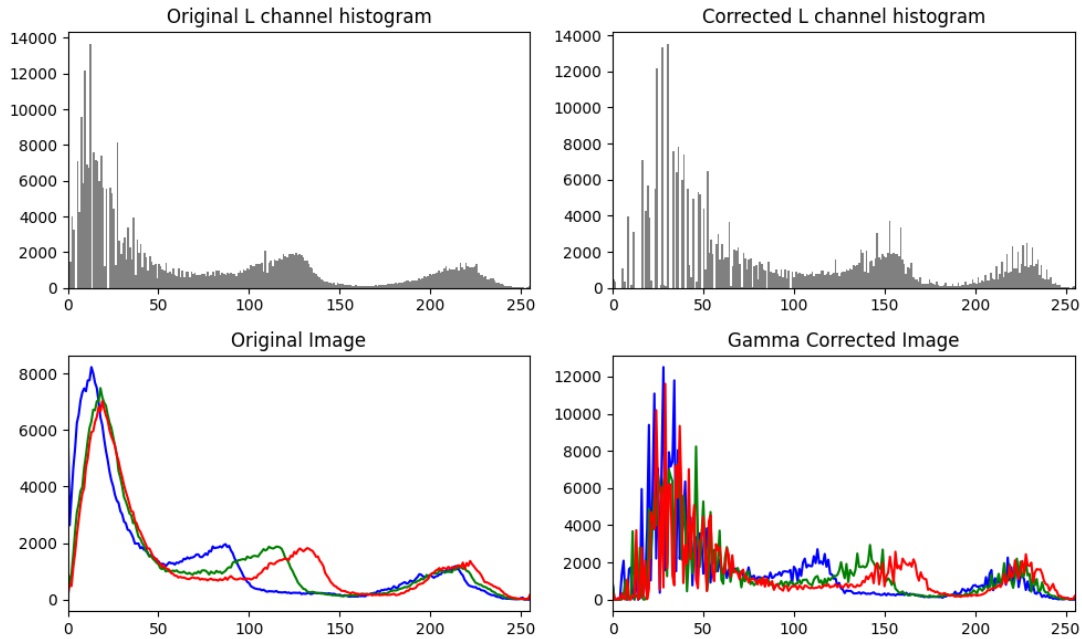
corrected = gamma_correction(img, gamma_lut(0.7))
L0 = cv.split(cv.cvtColor(img, cv.COLOR_BGR2LAB))[0]
L1 = cv.split(cv.cvtColor(corrected, cv.COLOR_BGR2LAB))[0]

fig, axes = plt.subplots(2,2, figsize=(10, 6))
axes[0,0].hist(L0.ravel(), bins=256, range=(0,255), color='gray')
axes[0,0].set_title('Original L channel histogram')
axes[0,1].hist(L1.ravel(), bins=256, range=(0,255), color='gray')
axes[0,1].set_title('Corrected L channel histogram')

for ax, img, title in [(axes[1,0], img, 'Original Image'), (axes[1,1], corrected, 'Gamma Corrected Image')]:
    for i, c in enumerate('bgr'):
        ax.plot(cv.calcHist([img], [i], None, [256], [0, 256]), color=c, label=f'{c.upper()} channel')
    ax.set_title(title)
    for ax in axes.ravel():
        ax.set_xlim(0, 255)
plt.tight_layout(); plt.show()

```





1.4 Question 4

Increasing the vibrance of a photograph is probably achieved by applying an intensity transformation such as

$$f(x) = \min \left(x + a \times 128 e^{-\frac{(x-128)^2}{2\sigma^2}}, 255 \right),$$

to the saturation plane, where x is the input intensity, $a \in [0, 1]$ and $\sigma = 70$.

- Split the image shown in Fig. 4 into hue, saturation, and value planes.
- Apply the aforementioned intensity transformation to the saturation plane.
- Adjust a to get a visually pleasing output. Report the value of a .
- Recombine the three planes.
- Display the original image, vibrance-enhanced image, and the intensity transformation.

Figure 4: Image for enhancing the vibrance.

```
[155]: img = cv.imread("figures/fig4.png", cv.IMREAD_COLOR)
assert img is not None

SIGMA = 70

def vibrance_lut(a, sigma=SIGMA):
    """ Create a lookup table for vibrance adjustment based on the given sigma value. """
    x = np.arange(256, dtype='float64')
    f = np.minimum(x + a*128*np.exp(-(x-128)**2/(2*sigma**2)), 255)
    return np.round(f).astype(np.uint8)

def enhance(img, a, sigma):
    """ Enhance the vibrance of an image using the specified parameters. """
    h, s, v = cv.split(cv.cvtColor(img, cv.COLOR_BGR2HSV))
    s = cv.LUT(s, vibrance_lut(a, sigma))
    enhanced = cv.merge([h, s, v])
    return cv.cvtColor(enhanced, cv.COLOR_HSV2BGR)

h, s, v = cv.split(cv.cvtColor(img, cv.COLOR_BGR2HSV))
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, plane, name in zip(axes, [h, s, v], ['Hue', 'Saturation', 'Value']):
    ax.imshow(plane, cmap='gray', vmin=0, vmax=255)
    ax.set_title(name)
```

```

    ax.axis('off')
plt.tight_layout(); plt.show()

for a in [0.2, 0.4, 0.6, 0.8, 1.0]:
    s2 = cv.LUT(s, vibrance_lut(a))
    print(f'a={a}: mean S {s.mean():.0f} -> {s2.mean():.0f}, clipped at 255: {100*(s2==255).mean():.1f}%')

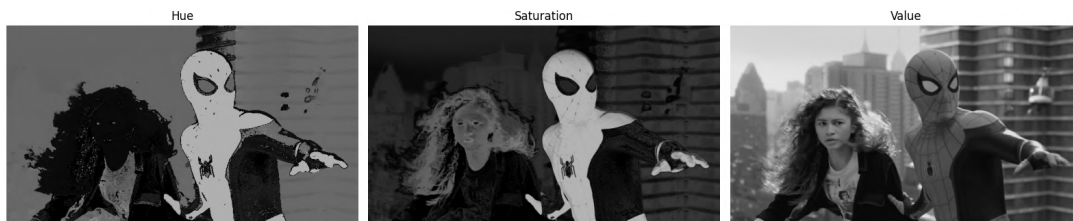
#picking a=0.7 for the final image

A = 0.7
out = enhance(img, A, SIGMA)

def img_draw2(img, vibrance_lut, A, out):
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))
    axes[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); axes[0].set_title('Original'); axes[0].axis('off')
    axes[1].imshow(cv.cvtColor(out, cv.COLOR_BGR2RGB)); axes[1].set_title(f'Vibrance, a = {A}'); axes[1].
    axis('off')
    axes[2].plot(vibrance_lut(A), label=f'a = {A}'); axes[2].plot(np.arange(256), '--', color='gray', lw=1,
    label='identity'); axes[2].set_xlim(0,255); axes[2].set_ylim(0,255); axes[2].set_aspect('equal')
    axes[2].set_xlabel(r'Input,  $\mathbf{x}$ '); axes[2].set_ylabel(r'Output,  $\mathbf{T}[\mathbf{x}]$ );
    axes[2].legend()

img_draw2(img, vibrance_lut, A, out)
plt.tight_layout(); plt.show()

```



```

a=0.2: mean S 51 -> 63, clipped at 255: 0.0%
a=0.4: mean S 51 -> 75, clipped at 255: 0.0%
a=0.6: mean S 51 -> 87, clipped at 255: 0.0%
a=0.8: mean S 51 -> 99, clipped at 255: 8.5%
a=1.0: mean S 51 -> 109, clipped at 255: 10.8%

```



1.5 Question 5

Write a function of your own to carry out histogram equalization on the image shown in Fig. 5. Show the histograms before and after equalization.

Figure 5: Image for histogram equalization.

```

[156]: from numpy import cumsum

img = cv.imread("figures/fig5.png", cv.IMREAD_GRAYSCALE)
assert img is not None

```

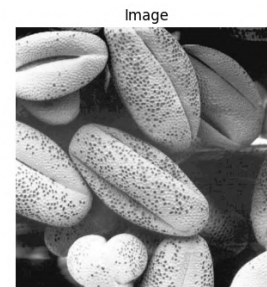
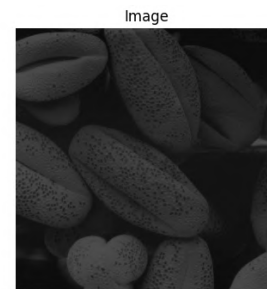
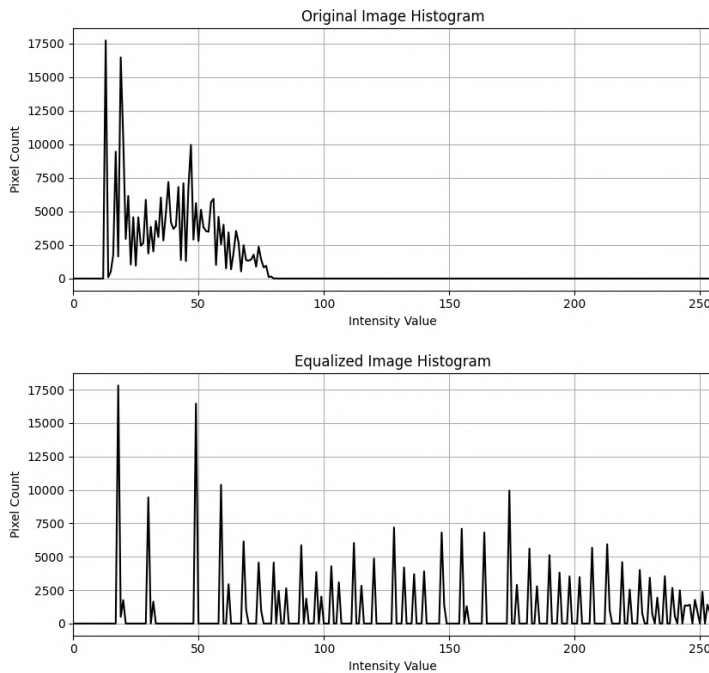
```
def plot_histogram(img, title='Histogram'):
    """ Plot the histogram of a grayscale image. """
    hist = cv.calcHist([img], [0], None, [256], [0, 256])
    fig, ax = plt.subplots(1,2, figsize=(15, 4))
    ax[0].plot(hist, color='black'); ax[0].set_title(title); ax[0].set_xlabel('Intensity Value'); ax[0].
    set_ylabel('Pixel Count'); ax[0].set_xlim([0, 255]); ax[0].grid()
    ax[1].imshow(img, cmap='gray', vmin=0, vmax=255); ax[1].set_title('Image'); ax[1].axis('off')
    plt.tight_layout()
    plt.show()

plot_histogram(img, title='Original Image Histogram')

def equalize_histogram(img, L=256):
    """ Perform histogram equalization on a grayscale image. """
    MN = img.size
    n_k = np.bincount(img.ravel(), minlength=L)
    cdf = cumsum(n_k)
    s_k = np.round((L-1) * cdf / MN).astype(np.uint8)
    out = cv.LUT(img, s_k)
    return out, s_k

equalized_img, s_k = equalize_histogram(img)

plot_histogram(equalized_img, title='Equalized Image Histogram')
```



1.6 Question 6

In this question, we will apply histogram equalization only to the foreground of an image to produce an image with a histogram equalized foreground.

- Open the image in Fig. 6, split it into hue, saturation, and values and display these planes in grayscale.
- Select the appropriate plane to threshold in extract the foreground mask. A mask is a binary image.
- Now obtain the foreground only using `cv.bitwise_and` and compute the histogram.
- Obtain the cumulative sum of the histogram using `np.cumsum`.
- Use the formulas in slides to histogram-equalize the foreground.
- Extract the background and add with the histogram equalized foreground.

Show the hue, saturation, and value plane, the mask, the original image, and the result with the histogram-equalized foreground.

Figure 6: Image for histogram equalizing the foreground.

```
[157]: img = cv.imread("figures/fig6.png", cv.IMREAD_COLOR)
assert img is not None

h, s, v = cv.split(cv.cvtColor(img, cv.COLOR_BGR2HSV))
fig, ax = plt.subplots(1, 3, figsize=(16, 5))
for a, plane, name in zip(ax, [h, s, v], ['Hue', 'Saturation', 'Value']):
    a.imshow(plane, cmap='gray', vmin=0, vmax=255)
    a.set_title(name)
    a.axis('off')
plt.tight_layout(); plt.show()

#sat channel is the choice for masking
ret, mask = cv.threshold(s, 13, 255, cv.THRESH_BINARY)
foreground = cv.bitwise_and(v, v, mask=mask)
hist = cv.calcHist([foreground], [0], mask, [256], [0, 256]).flatten()

fig, ax = plt.subplots(1, 3, figsize=(16, 3))
ax[0].plot(hist); ax[0].set_title('Histogram of Foreground'); ax[0].set_xlabel('Pixel Value'); ax[0].
    .set_ylabel('Frequency')
ax[1].imshow(mask, cmap='gray', vmin=0, vmax=255); ax[1].set_title('Mask'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB)); ax[2].set_title('Foreground'); ax[2].axis('off')
plt.tight_layout(); plt.show()

cdf = np.cumsum(hist)
cdf_normalized = cdf * 255 / cdf.max()

NM = np.count_nonzero(mask)
s_k = np.round((255 * cdf / NM).astype(np.uint8))
out_foreground = cv.LUT(foreground, s_k)
out_foreground = cv.bitwise_and(out_foreground, out_foreground, mask=mask)
inv_mask = cv.bitwise_not(mask)
background = cv.bitwise_and(v, v, mask=inv_mask)

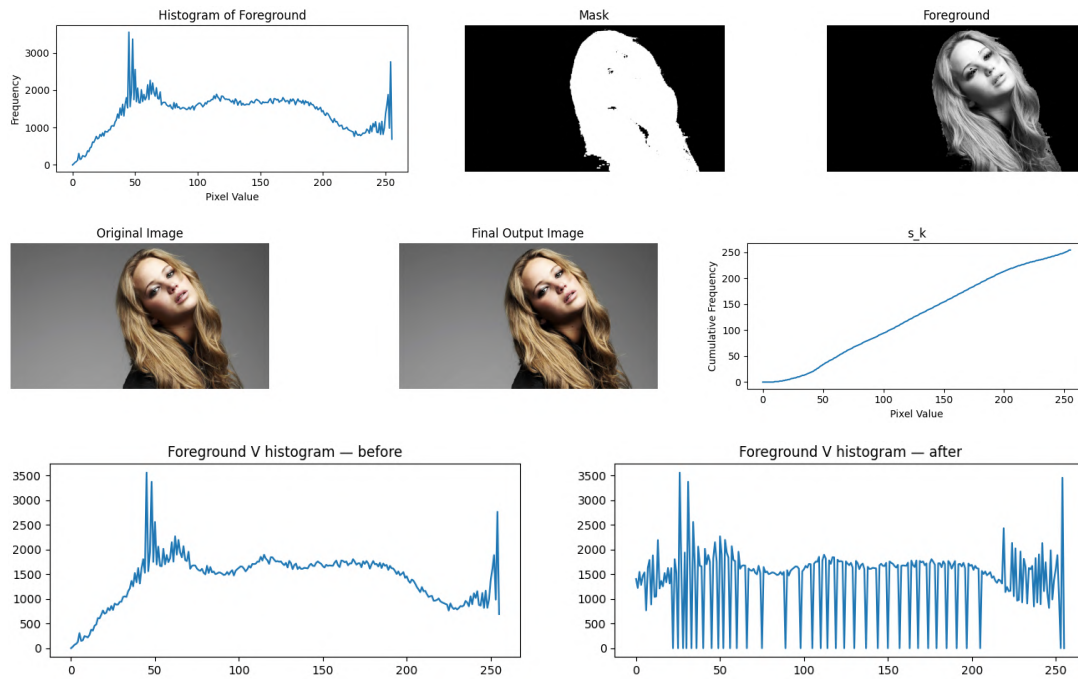
out_v = cv.add(out_foreground, background)
out = cv.merge([h, s, out_v])

fig, ax = plt.subplots(1, 3, figsize=(16, 3))
ax[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); ax[0].set_title('Original Image'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(out, cv.COLOR_HSV2RGB)); ax[1].set_title('Final Output Image'); ax[1].axis('off')
ax[2].plot(s_k); ax[2].set_title('s_k'); ax[2].set_xlabel('Pixel Value'); ax[2].set_ylabel('Cumulative Frequency')
plt.tight_layout(); plt.show()

hist_before = cv.calcHist([foreground], [0], mask, [256], [0, 256]).flatten()
hist_after = cv.calcHist([out_foreground], [0], mask, [256], [0, 256]).flatten()

fig, ax = plt.subplots(1, 2, figsize=(16, 3))
ax[0].plot(hist_before); ax[0].set_title('Foreground V histogram - before')
ax[1].plot(hist_after); ax[1].set_title('Foreground V histogram - after')
plt.show()
```





1.7 Question 7

Filtering with the Sobel operator can compute the gradient. Consider the image shown in Fig. 7

- Using the existing `filter2D` to Sobel filter the image.
- Write your own code to Sobel filter the image.
- Using the property

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \end{bmatrix},$$

carry out Sobel filtering.

Figure 7: Image for Sobel filtering.

```
[158]: img = cv.imread("figures/fig7.png", cv.IMREAD_GRAYSCALE)
assert img is not None
img_f = img.astype(np.float64)

Gx_kernel = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]], dtype=np.float64)
Gy_kernel = Gx_kernel.T

Gx = cv.filter2D(img_f, -1, Gx_kernel)
Gy = cv.filter2D(img_f, -1, Gy_kernel)

grad_map = np.sqrt(Gx**2 + Gy**2)
grad_map_disp = np.clip(grad_map / grad_map.max() * 255, 0, 255).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(18, 5))
ax[0].imshow(img, cmap='gray', vmin=0, vmax=255); ax[0].set_title('Original Image'); ax[0].axis('off')
ax[1].imshow(np.abs(Gx), cmap='gray', vmin=0, vmax=255); ax[1].set_title('Gx'); ax[1].axis('off')
ax[2].imshow(np.abs(Gy), cmap='gray', vmin=0, vmax=255); ax[2].set_title('Gy'); ax[2].axis('off')
ax[3].imshow(grad_map_disp, cmap='gray', vmin=0, vmax=255); ax[3].set_title('Gradient Magnitude'); ax[3].
    axis('off')
plt.tight_layout(); plt.show()

def conv2d_man(img, kernel):
    """ Perform 2D convolution on an image using a given kernel. """
```

```

kh, kw = kernel.shape
pad_h, pad_w = kh // 2, kw // 2
padded_img = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant', constant_values=0)

out = np.zeros_like(img, dtype=np.float64)
H, W = img.shape

for i in range(H):
    for j in range(W):
        region = padded_img[i:i+kh, j:j+kw]
        out[i, j] = np.sum(region * kernel)
return out

Gx_manual = conv2d_man(img_f, Gx_kernel)
Gy_manual = conv2d_man(img_f, Gy_kernel)
grad_map_manual = np.sqrt(Gx_manual**2 + Gy_manual**2)
grad_map_disp_manual = np.clip(grad_map_manual / grad_map_manual.max() * 255, 0, 255).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(18, 5))
ax[0].imshow(img, cmap='gray', vmin=0, vmax=255); ax[0].set_title('Original Image'); ax[0].axis('off')
ax[1].imshow(np.abs(Gx_manual), cmap='gray', vmin=0, vmax=255); ax[1].set_title('Gx'); ax[1].axis('off')
ax[2].imshow(np.abs(Gy_manual), cmap='gray', vmin=0, vmax=255); ax[2].set_title('Gy'); ax[2].axis('off')
ax[3].imshow(grad_map_disp_manual, cmap='gray', vmin=0, vmax=255); ax[3].set_title('Gradient Magnitude'); ax[3].
    axis('off')
plt.tight_layout(); plt.show()

smooth = np.array([[1], [2], [1]], dtype=np.float64)
deriv = np.array([[1, 0, -1]], dtype=np.float64)

Gx_sep = cv.filter2D(cv.filter2D(img_f, -1, smooth), -1, deriv)
Gy_sep = cv.filter2D(cv.filter2D(img_f, -1, smooth.T), -1, deriv.T)

grad_map_sep = np.sqrt(Gx_sep**2 + Gy_sep**2)
grad_map_disp_sep = np.clip(grad_map_sep / grad_map_sep.max() * 255, 0, 255).astype(np.uint8)

fig, ax = plt.subplots(1, 4, figsize=(18, 5))
ax[0].imshow(img, cmap='gray', vmin=0, vmax=255); ax[0].set_title('Original Image'); ax[0].axis('off')
ax[1].imshow(np.abs(Gx_sep), cmap='gray', vmin=0, vmax=255); ax[1].set_title('Gx'); ax[1].axis('off')
ax[2].imshow(np.abs(Gy_sep), cmap='gray', vmin=0, vmax=255); ax[2].set_title('Gy'); ax[2].axis('off')
ax[3].imshow(grad_map_disp_sep, cmap='gray', vmin=0, vmax=255); ax[3].set_title('Gradient Magnitude'); ax[3].
    axis('off')
plt.tight_layout(); plt.show()

```





1.8 Question 8

Write a program to zoom images by a given factor $s \in (0, 10]$. You must use a function to zoom the image, which can handle

- (a) nearest-neighbor, and
- (b) bilinear interpolation.

I have included four images, two large originals, and there zoomed-out versions. Test your algorithm by computing the normalized sum of squared difference (SSD) when you scale-up the given small images by a factor of 4 by comparing with the original images.

```
[159]: def zoom_image(img, s_x, s_y, method='nearest'):
    """ s: zoom factor, float in (0, 10], method: 'nearest' or 'bilinear' """
    if img.ndim == 2:
        img = img[:, :, np.newaxis]
        squeeze = True
    else:
        squeeze = False

    H, W, C = img.shape
    new_H, new_W = int(round(H * s_y)), int(round(W * s_x))

    ys = np.arange(new_H) / s_y
    xs = np.arange(new_W) / s_x

    if method == 'nearest':
        y_idx = np.clip(np.round(ys), 0, H - 1).astype(int); x_idx = np.clip(np.round(xs), 0, W - 1).astype(int)
        out = img[np.ix_(y_idx, x_idx)]

    elif method == 'bilinear':
        y0 = np.clip(np.floor(ys), 0, H - 1).astype(int); x0 = np.clip(np.floor(xs), 0, W - 1).astype(int)
        y1 = np.clip(y0 + 1, 0, H - 1); x1 = np.clip(x0 + 1, 0, W - 1)

        dy = (ys - y0).reshape(-1, 1, 1); dx = (xs - x0).reshape(1, -1, 1)

        Ia = img[np.ix_(y0, x0)]; Ib = img[np.ix_(y0, x1)]; Ic = img[np.ix_(y1, x0)]; Id = img[np.ix_(y1, x1)]

        top = Ia * (1 - dx) + Ib * dx; bottom = Ic * (1 - dx) + Id * dx
        out = top * (1 - dy) + bottom * dy

    out = np.clip(out, 0, 255).astype(np.uint8)

    if squeeze:
        out = out[:, :, 0]
    return out

def normalized_ssd(img1, img2):
    img1 = img1.astype(np.float64)
    img2 = img2.astype(np.float64)
    ssd = np.sum((img1 - img2) ** 2)
    norm_ssd = ssd / img1.size # normalize by number of pixels (or by max possible SSD, see note below)
    return norm_ssd

def show_scaled(small, large, f_x, f_y, method):
```

```

print(f"small: {small.shape}, large: {large.shape}")
scaled = zoom_image(small, f_x, f_y, method)
print("Normalized SSD:", normalized_ssd(scaled, large))

fig, ax = plt.subplots(1, 3, figsize=(16, 5))
ax[0].imshow(cv.cvtColor(small, cv.COLOR_BGR2RGB)); ax[0].set_title('Small Image'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(scaled, cv.COLOR_BGR2RGB)); ax[1].set_title(f'Scaled Image ({method})'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(large, cv.COLOR_BGR2RGB)); ax[2].set_title('Large Image'); ax[2].axis('off')
plt.tight_layout(); plt.show()

for i in 'abc':
    small = cv.imread(f"figures/fig8{i}_small.png", cv.IMREAD_COLOR)
    assert small is not None
    large = cv.imread(f"figures/fig8{i}_large.png", cv.IMREAD_COLOR)
    assert large is not None
    f_y = large.shape[0] / small.shape[0]
    f_x = large.shape[1] / small.shape[1]
    show_scaled(small, large, f_x, f_y, 'nearest')
    show_scaled(small, large, f_x, f_y, 'bilinear')

very_small = cv.imread('figures/fig8d_verySmall.jpg', cv.IMREAD_COLOR)
assert very_small is not None
small = cv.imread("figures/fig8d_small.jpg", cv.IMREAD_COLOR)
assert small is not None
large = cv.imread("figures/fig8d_large.jpg", cv.IMREAD_COLOR)
assert large is not None

show_scaled(very_small, large, large.shape[1]/very_small.shape[1], large.shape[0]/very_small.shape[0], 'nearest')
show_scaled(very_small, large, large.shape[1]/very_small.shape[1], large.shape[0]/very_small.shape[0], 'bilinear')
show_scaled(small, large, large.shape[1]/small.shape[1], large.shape[0]/small.shape[0], 'nearest')
show_scaled(small, large, large.shape[1]/small.shape[1], large.shape[0]/small.shape[0], 'bilinear')

```

small: (270, 480, 3), large: (1080, 1920, 3)
Normalized SSD: 255.29569331918725



small: (270, 480, 3), large: (1080, 1920, 3)
Normalized SSD: 200.24514017489713



small: (300, 480, 3), large: (1200, 1920, 3)
Normalized SSD: 64.62937832754629



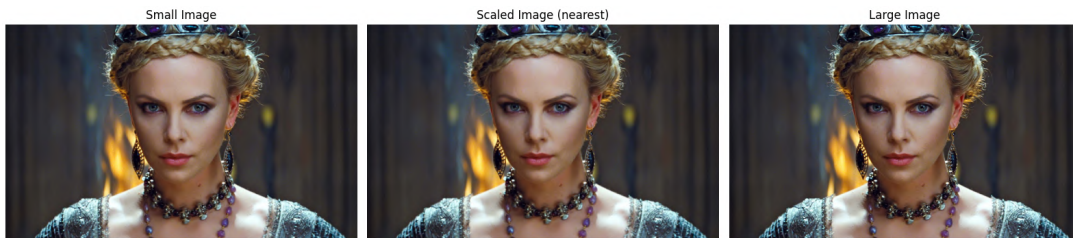
small: (300, 480, 3), large: (1200, 1920, 3)

Normalized SSD: 48.96066550925926



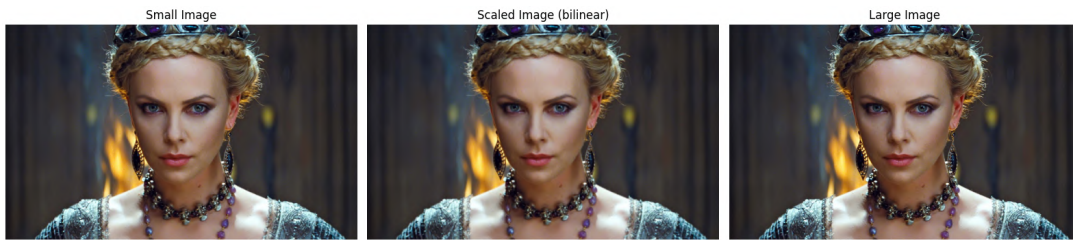
small: (365, 600, 3), large: (1459, 2400, 3)

Normalized SSD: 147.76545426852488



small: (365, 600, 3), large: (1459, 2400, 3)

Normalized SSD: 113.56613643286879



small: (28, 50, 3), large: (560, 1000, 3)

Normalized SSD: 737.8096303571429



small: (28, 50, 3), large: (560, 1000, 3)

Normalized SSD: 654.5981875



small: (112, 200, 3), large: (560, 1000, 3)
Normalized SSD: 320.0882172619048



small: (112, 200, 3), large: (560, 1000, 3)
Normalized SSD: 285.1395523809524



1.9 Question 9

Fig. 92 shows a flower image with both the foreground and background are in focus.

- (a) Use `grabCut` to segment the image. Show the final segmentation mask, foreground image, and background image.
- (b) Produce an enhanced image with a substantially blurred background. Display the original image alongside the enhanced image.
- (c) Why is the background just beyond the edge of the flower quite dark in the enhanced image?

2 <https://steemit.com/marguerite/@ctrl-alt-nwo/marguerite-daisy>

Figure 8: Image enhancing.

```
[160]: img = cv.imread("figures/fig9.png", cv.IMREAD_COLOR)
assert img is not None

mask = np.zeros(img.shape[:2], np.uint8)
bgd_model = np.zeros((1, 65), np.float64)
fgd_model = np.zeros((1, 65), np.float64)

h, w = img.shape[:2]
rect = (int(0.1*w), int(0.1*h), int(0.9*w), int(0.8*h))

cv.grabCut(img, mask, rect, bgd_model, fgd_model, iterCount=5, mode=cv.GC_INIT_WITH_RECT)

binary_mask = np.where((mask == cv.GC_FGD) | (mask == cv.GC_PR_FGD), 1, 0).astype('uint8')

foreground = img * binary_mask[:, :, np.newaxis]
background = img * (1 - binary_mask[:, :, np.newaxis])

fig, ax = plt.subplots(1, 4, figsize=(18, 5))
ax[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); ax[0].set_title('Original'); ax[0].axis('off')
ax[1].imshow(binary_mask, cmap='gray'); ax[1].set_title('Segmentation Mask'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB)); ax[2].set_title('Foreground'); ax[2].axis('off')
ax[3].imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB)); ax[3].set_title('Background'); ax[3].axis('off')
plt.tight_layout(); plt.show()

blurred_bg = cv.GaussianBlur(background, (35, 35), sigmaX=15)
mask_3ch = binary_mask[:, :, np.newaxis].astype(bool)
enhanced = np.where(mask_3ch, img, blurred_bg)

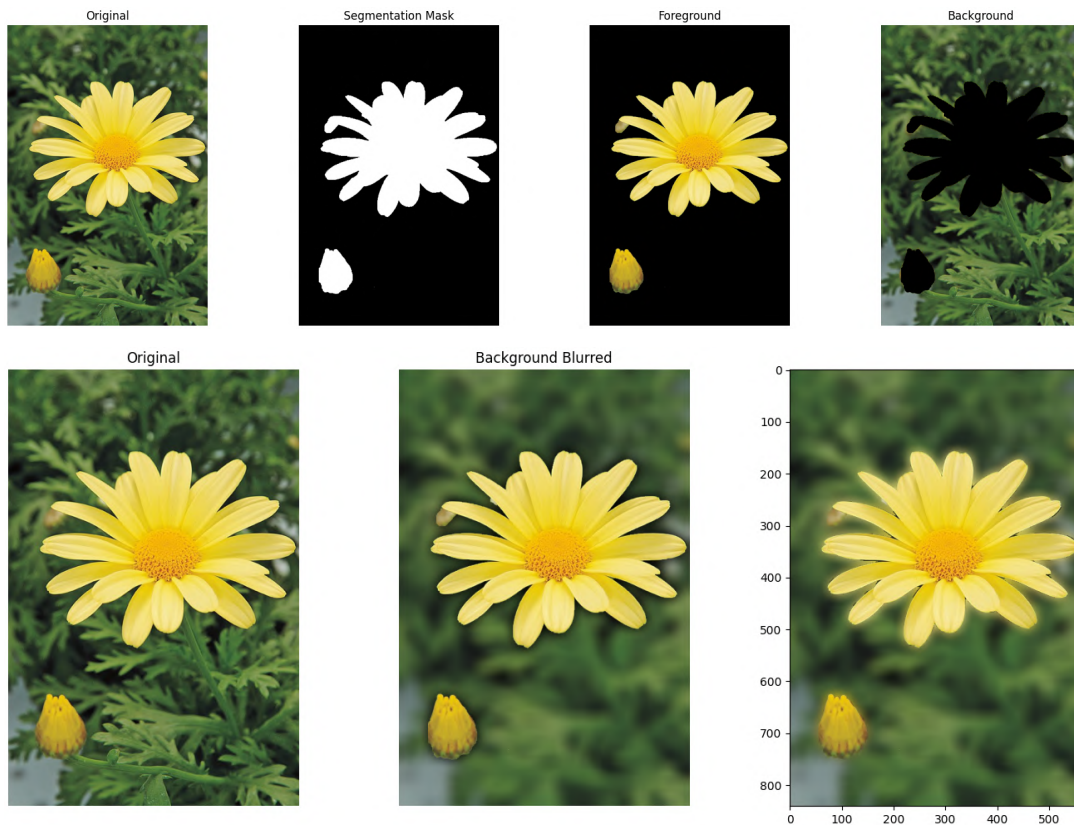
blurred_bg2 = cv.GaussianBlur(img, (35, 35), sigmaX=15)
```

```

enhanced2 = np.where(mask_3ch, img, blurred_bg2)

fig, ax = plt.subplots(1, 3, figsize=(14, 6))
ax[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); ax[0].set_title('Original'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(enhanced, cv.COLOR_BGR2RGB)); ax[1].set_title('Background Blurred'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(enhanced2, cv.COLOR_BGR2RGB)); ax[2].set_title('Background Blurred'); ax[2].axis('off')
plt.tight_layout(); plt.show()

```



Background is dark because in the extracted background the foreground is filled black, and when a gaussian blur is applied that black values are mixed in with the actual background values as well. This can be reduced if we use the whole image when blurring the background instead of the extracted background

1.10 Question 10

Fig. 10 shows an image of Lake Minaret. The requirement is to smoothen the image while keeping the edges sharp.

(a) Use OpenCV's `bilateralFilter` to smoothen the image with a bilateral filter with σ_s and σ_r of your choice. Display the original, Gaussian blurred image with a similar kernel, and bilateral filtered images side-by-side.

(b) Implement your own bilateral filter and compare the results with OpenCV's `bilateralFilter`. Display the original, Gaussian blurred image with a similar kernel, and bilateral filtered images side-by-side. Use a suitable quantitative measure to compare the results.

Figure 9: Image bilateral filtering.

```

[161]: img = cv.imread("figures/fig9.png", cv.IMREAD_COLOR)
assert img is not None

sigma_s = 15; sigma_r = 85

bilateral = cv.bilateralFilter(img, d=-1, sigmaColor=sigma_r, sigmaSpace=sigma_s)
ksize = 2 * int(3 * sigma_s) + 1
gaussian = cv.GaussianBlur(img, (ksize, ksize), sigmaX=sigma_s)

def bilateral_manual(img, sigma_s, sigma_r, radius=None):
    """ Manual Bilatarel Filter """

```

```

if radius is None:
    radius = int(3 * sigma_s)

img_f = img.astype(np.float64)

if img.ndim == 2:
    img_f = img_f[:, :, np.newaxis]
    squeeze = True
else:
    squeeze = False

H, W, C = img_f.shape
padded = np.pad(img_f, ((radius, radius), (radius, radius), (0, 0)), mode='reflect')
out = np.zeros_like(img_f)

ys, xs = np.mgrid[-radius:radius+1, -radius:radius+1]
spatial_weight = np.exp(-(xs**2 + ys**2) / (2 * sigma_s**2))

for i in range(H):
    for j in range(W):
        window = padded[i:i+2*radius+1, j:j+2*radius+1, :]
        center = img_f[i, j, :]
        diff = window - center
        range_weight = np.exp(-np.sum(diff**2, axis=2) / (2 * sigma_r**2))
        weight = spatial_weight * range_weight
        weight_sum = np.sum(weight)
        out[i, j, :] = np.sum(window * weight[:, :, np.newaxis], axis=(0,1)) / weight_sum

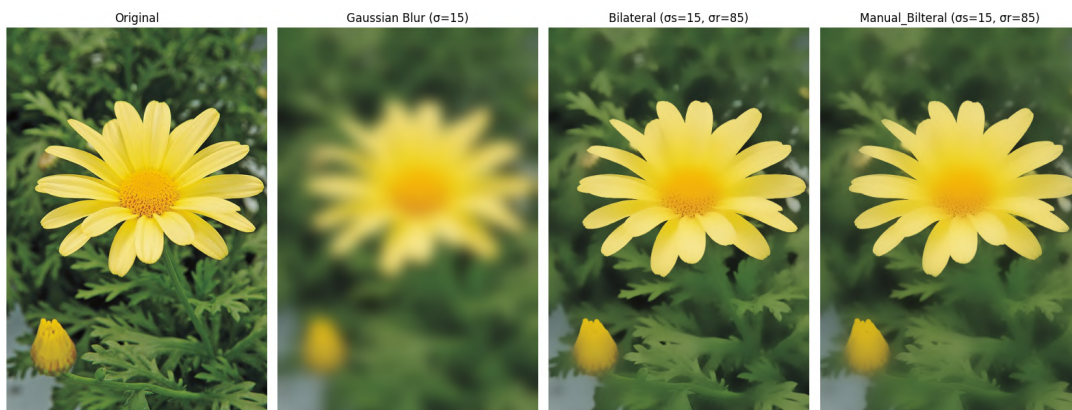
out = np.clip(out, 0, 255).astype(np.uint8)
if squeeze:
    out = out[:, :, 0]
return out

manual_bilateral = bilateral_manual(img, sigma_s, sigma_r)

fig, ax = plt.subplots(1, 4, figsize=(16, 6))
ax[0].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); ax[0].set_title('Original'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(gaussian, cv.COLOR_BGR2RGB)); ax[1].set_title(f'Gaussian Blur (σ={sigma_s})'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(bilateral, cv.COLOR_BGR2RGB)); ax[2].set_title(f'Bilateral (s={sigma_s}, r={sigma_r})'); ax[2].axis('off')
ax[3].imshow(cv.cvtColor(manual_bilateral, cv.COLOR_BGR2RGB)); ax[3].set_title(f'Manual_Bilateral (s={sigma_s}, r={sigma_r})'); ax[3].axis('off')
plt.tight_layout(); plt.show()

ssd_vs_cv = normalized_ssd(manual_bilateral, bilateral)
print("Normalized SSD vs cv.bilateralFilter:", ssd_vs_cv)

```



Normalized SSD vs cv.bilateralFilter: 31.817818670724876